LABORATORY PROGRAM – 7

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

```
import
org.apache.hadoop.conf.Configur
ation; import
org.apache.hadoop.fs.Path;

    import
    org.apache.hadoop.io.IntWrit
    able; import
    org.apache.hadoop.io.Text;

    import org.apache.hadoop.mapreduce.Job;
    import
    org.apache.hadoop.mapreduce.lib.input.FileIn
    putFormat; import
    org.apache.hadoop.mapreduce.lib.output.File
    OutputFormat;

    public class TopNDriver {

        public static void main(String[] args)
          throws Exception { if (args.length !=
          3) {
          System.err.println("Usage: TopNDriver <in> <temp-out>
          <final-out>"); System.exit(2);
        }

        Configuration conf = new Configuration();

        // === Job 1: Word Count ===
        Job wcJob = Job.getInstance(conf, "word
        count");
        wcJob.setJarByClass(TopNDriver.class);
        wcJob.setMapperClass(WordCountMappe
        r.class);
        wcJob.setCombinerClass(WordCountRed
        ucer.class);
        wcJob.setReducerClass(WordCountReduc
        er.class);
        wcJob.setOutputKeyClass(Text.class);
```

```
    wcJob.setOutputValueClass(IntWritable.cl
    ass);

    FileInputFormat.addInputPath(wcJob, new
    Path(args[0])); Path tempDir = new
    Path(args[1]);
    FileOutputFormat.setOutputPath(wcJob,
    tempDir);

      if
        (!wcJob.waitForCompl
        etion(true)) {
        System.exit(1);
    }

    // === Job 2: Top N ===
    Job topJob = Job.getInstance(conf, "top 10
    words");
    topJob.setJarByClass(TopNDriver.class);
    topJob.setMapperClass(TopNMapper.clas
    s);
    topJob.setReducerClass(TopNReducer.cla
    ss);
    topJob.setMapOutputKeyClass(IntWritable
    .class);
    topJob.setMapOutputValueClass(Text.clas
    s); topJob.setOutputKeyClass(Text.class);
    topJob.setOutputValueClass(IntWritable.cl
    ass);

    FileInputFormat.addInputPath(topJob,
    tempDir);
    FileOutputFormat.setOutputPath(topJob,
    new Path(args[2]));

    System.exit(topJob.waitForCompletion(true) ? 0 : 1);
  }
```

Mapper Code (WordCountMapper.java)

```
package samples.topn;

import
java.io.IOException;
import
java.util.StringTokeni
zer;
```

```
import
org.apache.hadoop.io.IntWrit
able; import
org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper
  extends Mapper<Object, Text, Text, IntWritable> {

  private final static IntWritable ONE = new
  IntWritable(1); private Text word = new
  Text();
  // characters to normalize into spaces
  private String tokens = "[_|$#<>\\^=\\[\\]\\*/\\\\,;,.\\-:()?!\"']";

  @Override
    protected void map(Object key, Text value,
      Context context) throws IOException,
      InterruptedException {

    // clean & tokenize
    String clean = value.toString()
                .toLowerCase()
                .replaceAll(tokens, " ");
    StringTokenizer itr = new
    StringTokenizer(clean); while
    (itr.hasMoreTokens()) {
      word.set(itr.nextToken().
      trim());
      context.write(word,
      ONE);
    }
  }
}
```

Mapper Code (TopNMapper.java)

```
package

samples.topn;

import

java.io.IOExcepti

on;
```

```
import
org.apache.hadoop.io.IntWrit
able; import
org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class TopNMapper
  extends Mapper<Object, Text, IntWritable, Text> {

  private IntWritable count = new
  IntWritable(); private Text word =
  new Text();

  @Override
    protected void map(Object key, Text value,
      Context context) throws IOException,
      InterruptedException {

    // input line: word \t count
    String[] parts =
    value.toString().split("\\t"); if
    (parts.length == 2) {
      word.set(parts[0]);
      count.set(Integer.parseI
      nt(parts[1]));
      // emit count → word, so Hadoop sorts
      by count context.write(count, word);
    }
  }
}
```

Reducer Code (WordCountReducer.java)

```
package

samples.topn;

import

java.io.IOExcepti

on;



import
org.apache.hadoop.io.IntWrit
able; import
org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
```

```
public class WordCountReducer
  extends Reducer<Text, IntWritable, Text, IntWritable> {

  @Override
    protected void reduce(Text key, Iterable<IntWritable>
      values, Context context) throws IOException,
      InterruptedException {

    int sum = 0;
      for (IntWritable val
        : values) { sum
        += val.get();
    }
    context.write(key, new IntWritable(sum));
  }
}
```

Reducer Code (TopNReducer.java)

```
package samples.topn;

import
java.io.IOException;
import
java.util.ArrayList;
import
java.util.Collections;
import java.util.List;
import
java.util.Map;
import
java.util.TreeMa
p;

import
org.apache.hadoop.io.IntWrit
able; import
org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class TopNReducer
  extends Reducer<IntWritable, Text, Text, IntWritable> {

  // TreeMap with descending order of
  keys (counts) private
  TreeMap<Integer, List<String>>
  countMap =
    new TreeMap<>(Collections.reverseOrder());

  @Override
```

```
  protected void reduce(IntWritable key, Iterable<Text>
    values, Context context) throws IOException,
    InterruptedException {

  int cnt = key.get();
  List<String> words = countMap.getOrDefault(cnt,
  new ArrayList<>()); for (Text w : values) {
    words.add(w.toString());
  }
  countMap.put(cnt, words);
}

@Override
  protected void
    cleanup(Context context)
    throws IOException,
    InterruptedException {

  // collect top 10 word→count
  pairs List<WordCount> topList =
  new ArrayList<>(); int seen = 0;
    for (Map.Entry<Integer, List<String>> entry :
      countMap.entrySet()) { int cnt =
      entry.getKey();
      for (String w :
        entry.getValue()) {
        topList.add(new
        WordCount(w, cnt));
        seen++;
      if (seen == 10) break;
    }
    if (seen == 10) break;


  }

  // sort these 10 entries alphabetically by word
  Collections.sort(topList, (a, b) ->
  a.word.compareTo(b.word));

  // emit final top 10 in
  alphabetical order for
  (WordCount wc : topList) {
    context.write(new Text(wc.word), new IntWritable(wc.count));
  }
}

// helper class
```

```
    private static class
      WordCount {
      String word;
    int count;
    WordCount(String w, int c) { word = w; count = c; }
  }
}
```

```
┌──(thilak㉿kali)-[~]
└─$ mkdir top10
cd top10

┌──(thilak㉿kali)-[~/top10]
└─$ nano TopDriver.java

┌──(thilak㉿kali)-[~/top10]
└─$ nano TopMapper.java

┌──(thilak㉿kali)-[~/top10]
└─$ nano TopReducer.java

┌──(thilak㉿kali)-[~/top10]
└─$ javac -classpath $(hadoop classpath) -d . *.java
hadoop: command not found
error: invalid flag: .
Usage: javac <options> <source files>
use --help for a list of possible options

┌──(thilak㉿kali)-[~/top10]
└─$ export HADOOP_HOME=/home/thilak/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin

┌──(thilak㉿kali)-[~/top10]
└─$ javac -classpath $(hadoop classpath) -d . *.java

┌──(thilak㉿kali)-[~/top10]
└─$ jar -cvf top10.jar *
added manifest
adding: TopDriver.class(in = 1343) (out= 743)(deflated 44%)
adding: TopDriver.java(in = 893) (out= 367)(deflated 58%)
adding: TopMapper.class(in = 1709) (out= 714)(deflated 58%)
adding: TopMapper.java(in = 551) (out= 291)(deflated 47%)
adding: TopReducer.class(in = 3664) (out= 1513)(deflated 58%)
adding: TopReducer.java(in = 1199) (out= 504)(deflated 57%)

┌──(thilak㉿kali)-[~/top10]
└─$
```

```
                HDFS: Number of bytes written=44
                HDFS: Number of read operations=15
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=4
                HDFS: Number of bytes read erasure-coded=0
        Map-Reduce Framework
                Map input records=4
                Map output records=13
                Map output bytes=125
                Map output materialized bytes=157
                Input split bytes=105
                Combine input records=0
                Combine output records=0
                Reduce input groups=5
                Reduce shuffle bytes=157
                Reduce input records=13
                Reduce output records=5
                Spilled Records=26
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=15
                Total committed heap usage (bytes)=434110464
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=73
        File Output Format Counters
                Bytes Written=44

┌──(thilak㉿kali)-[~/top10]
└─$ hdfs dfs -cat /topoutput/part-r-00000
analytics       1
big     4
data    5
hadoop  2
science 1

┌──(thilak㉿kali)-[~/top10]
└─$
```